

Zach Breene, C. Wyatt Polasek

A3 Design Document: Part A - Design

10/20/23

Part A:

1. What problem are you trying to solve?

We are trying to solve the problem of reading an image stored in a specific format (ppm) into a two-dimensional array and applying transformations such as rotation, flipping, and transposition to this image.

2. What example inputs will help illuminate the problem?

- Filename of the image
- Name of the destination array to store pixel data
- Type of transformation to be performed (Rotate, Flip, Transposition)
- Rotation: The degree of rotation that should be applied to the image
- Flipping: The axis (x or y) in which the image should be flipped over
- Transpose: Specifically
- 2D array access method (row-major, col-major)

3. What example outputs go with those example inputs?

- The image with the applied transformations
- The image stored into the 2D array with the chosen access method

4. Into what steps or subproblems can you break down the problem?

1. Read data from image file
2. Write pixel data into 2D array
3. Choose a transformation included with necessary parameters
4. Apply transformations to each pixel using given parameters
5. Saving the modified pixel data to a new 2D array
6. Outputting the transformed image to the Display

5. What data are in each subproblem?

1. Image filename
2. Destination 2D array name
3. Name of transformation type and parameters
 - a. Degree of rotation
 - b. Axis to flip over

- 4. Pixels of image stored into the 2D array
(5&6). New 2D array for transformed pixels

6. What code or algorithms go with that data?

- (1&2): Code to iterate through the ppm file and write it to a 2D array
- (3&4): Algorithms to apply each type of transformations to the pixels
- (5&6): Code to reformat the transformed 2D array back into a ppm image

7. What abstractions will you use to help solve the problem?

- We will use the Array2 abstraction to represent the 2D array of pixels read from a ppm.
- We will use transformation and iterator modules to iterate through the image and apply transformation algorithms to the pixels in the image.

8. If you have to create new abstractions, what are their design checklists?

- Transformation Module: Holds the logic for different transformations like rotation, flipping, and transposition.
 - rotate(image: &Array2<Pixel>, degrees: i32) -> Array2<Pixel>
 - 0 degrees
 - 90 degrees
 - 180 degrees
 - 270 degrees
 - flip(image: &Array2<Pixel>, axis: Axis) -> Array2<Pixel>
 - X-axis
 - Y-axis
 - transpose(image: &Array2<Pixel>) -> Array2<Pixel>
 - UL-to-LR axis
- Iterator Module: Defines iterators for row-major and column-major traversal.
 - readImage(filename: &str, array2: &mut Array2<Pixel>, method: AccessMethod)
 - Col-major
 - Row-major

9. [*] What invariant properties should hold during the solution of the problem?

- Dimensions of the array after a transformation should remain the same.
- Each element represents a valid pixel (with proper color codes).
- Pixel values should not be modified, only reordered relative to the given transformation type.
- The dimensions of the image must be positive
- Rotation: Given a pixel at coordinates (x, y), a 90-degree clockwise rotation will place it at (y, -x), a 180-degree rotation at (-x, -y), and a 270-degree rotation at (-y, x).

- Flipping: Given a pixel at coordinates (x, y), flipping horizontally will place it at (-x, y), and flipping vertically will place it at (x, -y).
- Transposition: Given a pixel at coordinates (x, y), transposing will place it at (y, x).

10. What algorithms might help solve the problem?

- Algorithms to read and parse ppm files.
- Algorithms for rotating, flipping, and transposing elements of a 2D array.

And once you have a design,

11. [*] What are the major components of your program, and what are their interfaces? Components include functions as well as abstract data types. An interface includes contracts as well as function prototypes.

1. **Array2:** Represents 2D array of Pixels from a ppm image file

Interfaces:

- a. `readImage(filename: &str, array2: &mut Array2<Pixel>, method: AccessMethod):` Reads pixels from a chosen image file and writes it into a 2D array using the chosen access method.

Contract:

- i. The filename should be valid.
- ii. 'array2' should already be initialized to hold pixel data.
- iii. The dimensions of 'array2' should equal the resolution of the image
- iv. 'method' should specify the use of either col-major or row-major

2. **Transformation Module:** Holds and Applies logic for each transformation

Interfaces:

- a. `rotate(image: &Array2<Pixel>, degrees: i32) -> Array2<Pixel>:` Rotates the image by the chosen degrees.

Contract:

- i. 'image' should be a valid 2D array.
- ii. Valid input for 'degrees' can be 0, 90, 180, or 270.
- iii. The output dimensions should remain same as the input

- b. `flip(image: &Array2<Pixel>, axis: Axis) -> Array2<Pixel>:` Flips the image over the chosen axis (X or Y).

Contract:

- i. 'image' should be a valid 2D array.
- ii. 'axis' should be either X or Y.
- iii. The output dimensions should remain same as the input

- c. `transpose(image: &Array2<Pixel>) -> Array2<Pixel>:` Transposes the image across the UL-to-LR diagonal axis.

Contract:

- i. 'image' should be a valid 2D array.
- ii. Pixels will be transposed across the diagonal

- iii. The output dimensions should remain same as the input
- 3. **Iterator Module:** Method for iterating over a 2D array
 - Interfaces:**
 - a. `iter_row_major(&self) -> Array2RowMajor<T>`: Iterators over pixels in 'array2' in row-major order.
 - Contract:**
 - i. Should iterate to the end of each column in a row before moving to the next row.
 - b. `iter_col_major(&self) -> Array2ColMajor<T>`: Iterators over pixels in 'array2' in col-major order.
 - Contract:**
 - i. Should iterate to the end of each row in a column before moving to the next column.

12. [*] How do the components in your program interact? That is, what is the architecture of your program?

The ppmtrans program would be structured to parse the input, determine the required transformation, and then call the appropriate function(s) from the Transformation Module. The Iterator Module would first be called to move through the ppm image file and read the data into a 2D array. Then the Transformation Module would select the function corresponding to the chosen transformation type. Each transformation function holds the logic that will be applied to each pixel in the image. The Transformation module would then utilize the Iterator Module to traverse the array where the function is applied to each pixel. After the transformation is applied, the transformed pixel is sent to a new 2D array.

13. [*] What test cases will you use to convince yourself that your program works?

- 1. **Read/Write:** Read image to 2D array then redisplay image with no transformations
 - a. **Successful Output:** The same image, completely unchanged.
- 2. **Rotate:** Rotate image 0 degrees / 90 degrees / 180 degrees / 270 degrees
 - a. **Successful Output:** Image is rotated by the chosen number of degrees
- 3. **Flip:** Flip image over X-axis / Y-axis
 - a. **Successful Output:** Image is flipped over the chosen axis
- 4. **Transpose:** Transpose image across UL-to-LR axis
 - a. **Successful Output:** Image is transposed across the UL-to-LR axis (diagonally)

14. [*] What arguments will you use to convince a skeptical audience that your program works? [A~narrative description of an algorithm is \emph{not} an argument! A~convincing argument usually involves invariants that hold during execution and reasoning that once execution is over, the invariant implies the desired result.]

Our program strictly adheres to key invariants: the dimensions of Array2 always remain constant, pixel color codes are reordered but never altered, and every element in Array2 represents a valid pixel. The transformations, calculated using known mathematical principles, help to ensure mathematical consistency. For instance, a 90-degree rotation will move a pixel from (x, y) to $(y, -x)$. When moving through the image/array, our program allows the option of col-major and row-major to ensure consistency during iteration. The modularity of our program helps to ensure its functionality as any issues with the code can be easily traced to the source of the problem. We have also included test cases to validate the accuracy of each module. Ultimately we can argue that our program works because we have implemented various fail-safes across the program, ensuring that any issues would become apparent and could be easily revised.